Contents

Hai đường tới một tấm pin **1**
Thuật toán và pipeline định vị lỗi tấm pin từ ảnh nhiệt drone . . . 1

1. Bài toán và dữ liệu **2**
1.1 Đầu vào . . . 2
1.2 Đầu ra . . . 2
1.3 Vì sao phải dùng cả hai loại ảnh . . . 2

2. Toàn cảnh pipeline **3**

3. Ba bước chuẩn bị **3**
3.1 Hiệu chuẩn H — cây cầu giữa hai con mắt . . . 3
3.2 Lọc ảnh — bỏ ảnh xấu trước khi ghép . . . 7
3.3 Ghép ortho nhiệt — đi ngược chiều . . . 7

4. Model phát hiện lỗi **12**

5. Nhánh A — detect trên ảnh lẻ rồi chiếu **12**
5.1 Chuỗi chiếu: từ một pixel tới một điểm trên mặt đất . . . 12
5.3 Gán vào polygon tấm pin . . . 16
5.2 Gộp trùng, và trường n . . . 17

6. Nhánh B — quét thẳng trên ortho **19**

7. Phép so hai nhánh **20**
7.1 Bốn điều kiện . . . 20
7.2 Thủ phạm là hướng cảnh . . . 21
7.3 Kiểm chứng ngược: nét không phải là tương phản . . . 23

8. Đánh giá bằng nhãn tay **23**

9. Sai số định vị **24**

10. Kết luận **24**

Phụ lục A — Các con số không được trích **24**

Phụ lục B — Script và file kết quả **25**
Chạy lại toàn bộ . . . 25
Chạy lại phép so hai nhánh . . . 25
File kết quả . . . 25

Hai đường tới một tấm pin

Thuật toán và pipeline định vị lỗi tấm pin từ ảnh nhiệt drone

Dự án: 1008-stage1-rgb-thermal · **Ngày:** 17/08/2026 · **Dữ liệu:** trang trại Thuận Nam, DJI ZH20T, bay 31 m

Tài liệu này mô tả toàn bộ chuỗi xử lý — từ thẻ nhớ drone tới một dòng *"lỗi X nằm ở tấm D20-T131, toạ độ (lon, lat)"* — và trình bày phép so trực tiếp giữa **hai cách nhận diện**: detect trên từng ảnh lẻ rồi chiếu, so với ghép ortho trước rồi mới detect.

Bản slide tương ứng: `tai_lieu/slide_thuat_toan.html`.

---

1. Bài toán và dữ liệu

Camera nhiệt thấy vệt nóng, nhưng vệt nóng nằm ở pixel số mấy của ảnh nào thì không ai đi thay được. Việc của pipeline là biến pixel thành địa chỉ.

1.1 Đầu vào

Ảnh drone	**598 cặp** — `_W.JPG` RGB 4056×3040 và `_T.JPG` nhiệt 640×512, chụp đồng thời
Sản phẩm Metashape	`cameras.xml` (tư thế camera), DSM 12677×7497 float32, ortho màu 25354×14993 ở **0,957 cm/px**
Chất lượng GPS	Chuyến bay **RTK fix** — 598/598 ảnh có `RtkFlag = 50`, độ lệch ngang 0,83 cm

1.2 Đầu ra

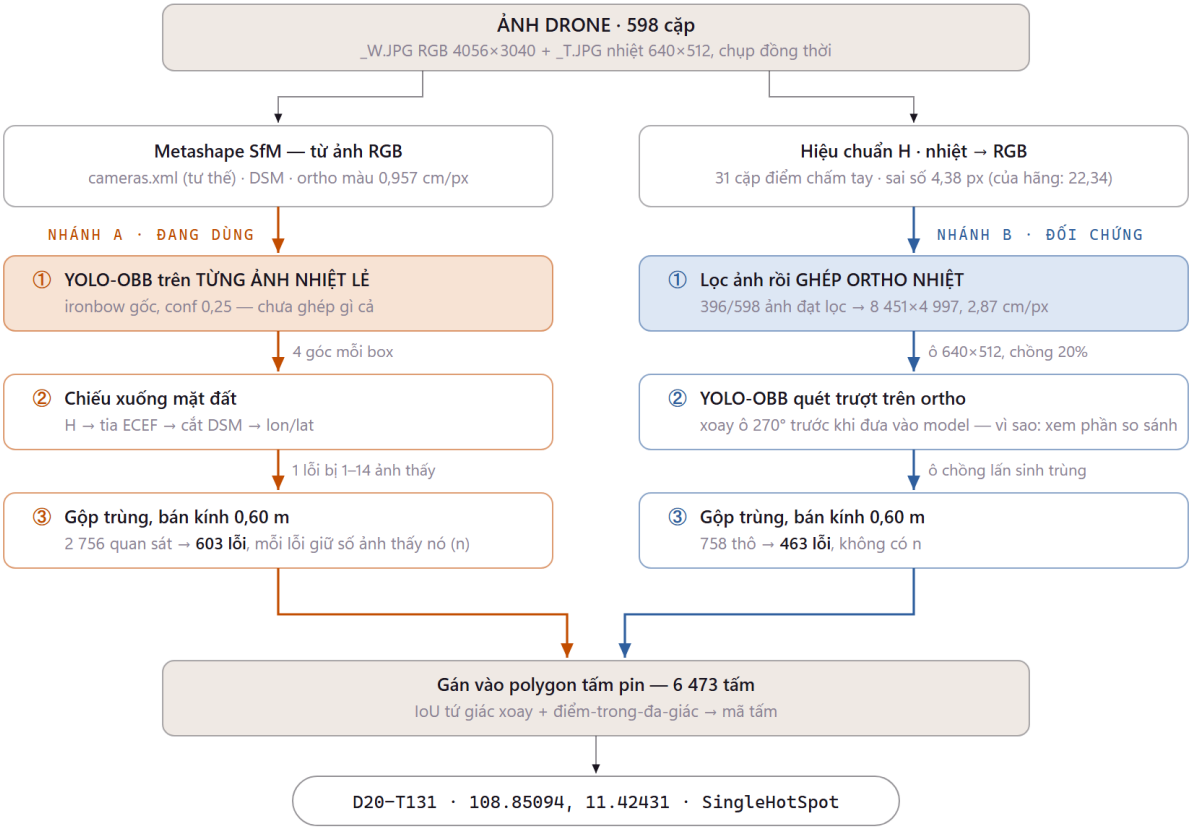
Một dòng cho mỗi lỗi: **lớp lỗi · lon/lat · mã tấm pin** dạng `D20-T131`, kèm ảnh cắt đối chứng để người soi xác nhận trước khi cử thợ.

Bốn lớp của model: `SingleHotSpot`, `MultiHotSpot`, `Bypass`, `ModuleOpenCircuit`. Lớp thứ tư ra **0 phát hiện** trên bộ Thuận Nam, nên mọi bảng thống kê chỉ thấy ba lớp.

1.3 Vì sao phải dùng cả hai loại ảnh

Điểm nóng chỉ hiện trên kênh nhiệt. Nhưng ảnh nhiệt 640×512 quá thô và quá ít đặc trưng để dựng SfM — Metashape không align nổi. Nên **hình học lấy từ RGB, tín hiệu lấy từ nhiệt**. Cây cầu nối hai bên là ma trận đồng dạng phối cảnh H.

2. Toàn cảnh pipeline



*Hình 1 — cả pipeline trong một hình. Hai nhánh dùng **cùng một model** `fault_obb_4class.pt` và **cùng một bước gán tấm**; chúng chỉ khác thứ tự của "ghép" và "detect". Phần chuẩn bị phía trên được cả hai nhánh dùng chung — nhánh A dùng để chiếu, nhánh B dùng để ghép.*

---

3. Ba bước chuẩn bị

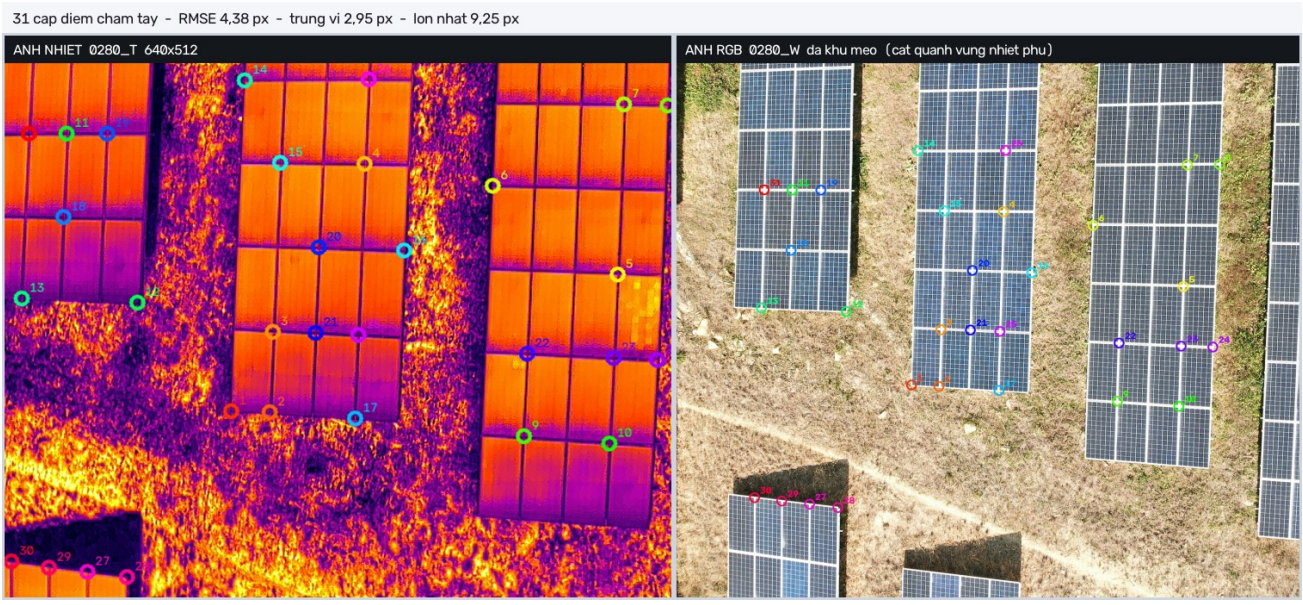
3.1 Hiệu chuẩn H — cây cầu giữa hai con mắt

Ống kính nhiệt và ống kính màu nằm cạnh nhau trên cùng một gimbal, nhìn cùng một cảnh nhưng không cùng một khung: lệch tâm quang, khác tiêu cự, khác độ phân giải. Một phép đồng dạng phối cảnh 3×3 nối chúng lại.

$$\begin{pmatrix} u' \\ v' \\ w \end{pmatrix} = \mathbf{H} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}, \qquad u_{rgb} = \frac{u'}{w}, \quad v_{rgb} = \frac{v'}{w}$$

Bước 1 — chấm 31 cặp điểm

Đây là **toàn bộ đầu vào** của phép hiệu chuẩn. Không có gì khác.



Hình 2 — 31 cặp điểm chấm tay trên cặp ảnh `0280`. Mỗi số ứng với một cặp: điểm bên trái trên ảnh nhiệt, điểm cùng số bên phải trên ảnh RGB. Chọn góc tấm pin và giao lối đi vì đó là chỗ nhìn rõ trên cả hai kênh — mặt tấm thì nhiệt gần như phẳng màu, không bám vào đâu được.

Khớp bằng bình phương tối thiểu trên cả 31 điểm (`fit_method: least_squares_all_points`). Kết quả:

Sai số tái chiếu RMSE	**4,38 px**
Trung vị	2,95 px
Lớn nhất	9,25 px
Quy ra mặt đất	4,35 cm

Ma trận thu được, ghi thẳng từ `H_thermal_to_rgb`:

$$\mathbf{H} = \begin{pmatrix} 2{,}639818 & 0{,}025512 & 1195{,}017 \\ -0{,}015879 & 2{,}633711 & 961{,}633 \\ 1{,}1181 \times 10^{-5} & 1{,}7799 \times 10^{-6} & 1 \end{pmatrix}$$

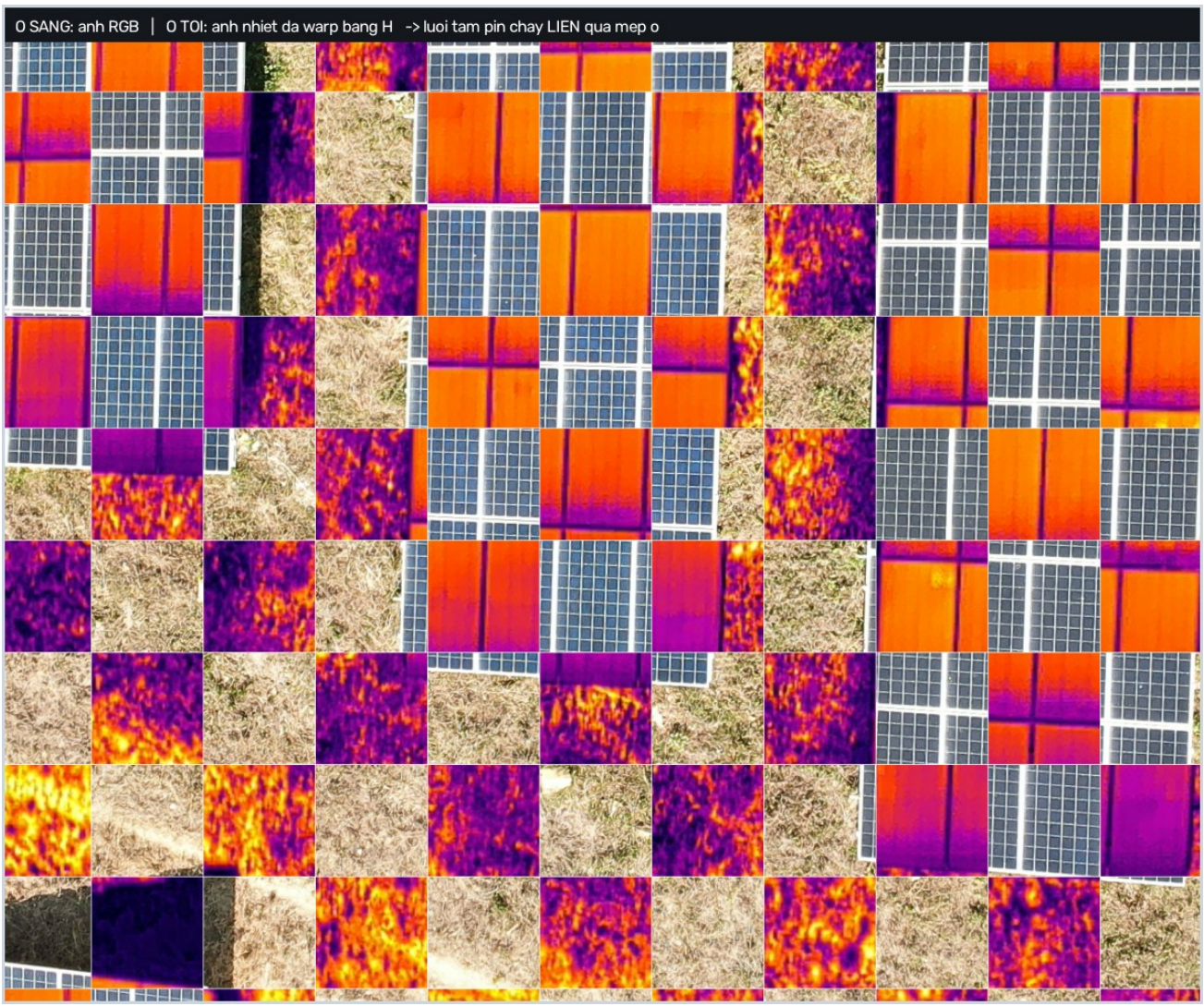
Bốn khối của nó đọc ra bốn thứ khác nhau:

khối	giá trị	nói lên điều gì
chéo chính 2×2	2,6398 · 2,6337	**phóng đại** — 1 px nhiệt trải thành 2,637 px RGB ($\sqrt{\det}$). Vì thế khung nhiệt 640 px chỉ phủ **41%** bề ngang khung RGB 4056 px
ngoài chéo 2×2	0,0255 · −0,0159	**xoay** ~0,3–0,55° giữa hai cảm biến, kèm một chút méo trượt — hai ống kính không gắn thẳng hàng tuyệt đối
cột 3	1195,0 · 961,6	**tịnh tiến** — góc trái-trên ảnh nhiệt rơi vào pixel (1195, 962) của ảnh RGB

khối	giá trị	nói lên điều gì
hàng 3	$\sim 10^{-5}$	**phối cảnh**, rất nhẹ: w chỉ đổi 0,7% từ mép trái sang mép phải. Hai cảm biến gần như song song, H gần với một phép đồng dạng

Một cách kiểm nhanh bằng số: tâm ảnh nhiệt (320, 256) qua H rơi vào (2038, 1624) trên ảnh RGB, trong khi tâm ảnh RGB là (2028, 1520). Lệch 10 px ngang nhưng **104 px dọc** — đúng chiều hai ống kính xếp trên dưới trên gimbal.

Bước 2 — H làm gì



*Hình 3 — kiểm tra bằng bàn cờ. Ô sáng là ảnh RGB, ô tối là ảnh nhiệt sau khi warp bằng H. Mép tấm pin và lối đi **chạy liền qua đường kẻ trắng** — đó là dấu hiệu H đúng. Nếu H sai, mỗi lần vượt mép ô là hình bị gãy khúc.*

Bước 3 — có H rồi thì dùng vào đâu

Đây là chỗ H trả công. Metashape chỉ giải tư thế và nội tham số cho **kênh RGB** — 598 camera, một bộ `f, cx, cy, k1…k3` duy nhất. Không có bài toán nào giải riêng cho kênh nhiệt, và cũng không cần: H là cửa để một

pixel nhiệt bước vào đúng mô hình camera RGB đó, rồi từ đó đi xuống mặt đất. Cả hai nhánh của pipeline đều đi qua cửa này, chỉ khác chiều.

Chiều xuôi — H — detect trước, chiếu sau (`ma/quet_loi_anh_le.py`, hàm `chieu`):

$$\text{px ảnh nhiệt} \xrightarrow{\mathbf{H}} \text{px RGB} \rightarrow \text{tia trong hệ camera} \rightarrow \text{tia ECEF} \rightarrow \text{cắt DSM} \rightarrow \text{lon/lat}$$

Model chạy trên ảnh nhiệt gốc, ra hộp lỗi theo pixel nhiệt; H đưa hộp đó sang hệ RGB để chuỗi chiếu của thầy dùng lại nguyên vẹn.

Chiều ngược — $\mathbf{H}^{-1}$ — dựng ortho nhiệt (`ma/ghep_ortho_nhiet.py`, `ma/bu_theo_rgb.py`):

$$\text{px ortho} \rightarrow \text{lon/lat} \rightarrow \text{cao độ DSM} \rightarrow \text{điểm 3D} \rightarrow \text{chiếu vào ảnh RGB} \xrightarrow{\mathbf{H}^{-1}} \text{px ảnh nhiệt} \rightarrow \text{lấy giá trị}$$

Mỗi pixel ortho hỏi ngược về một pixel nhiệt để lấy màu. Không có H thì không có ortho nhiệt.

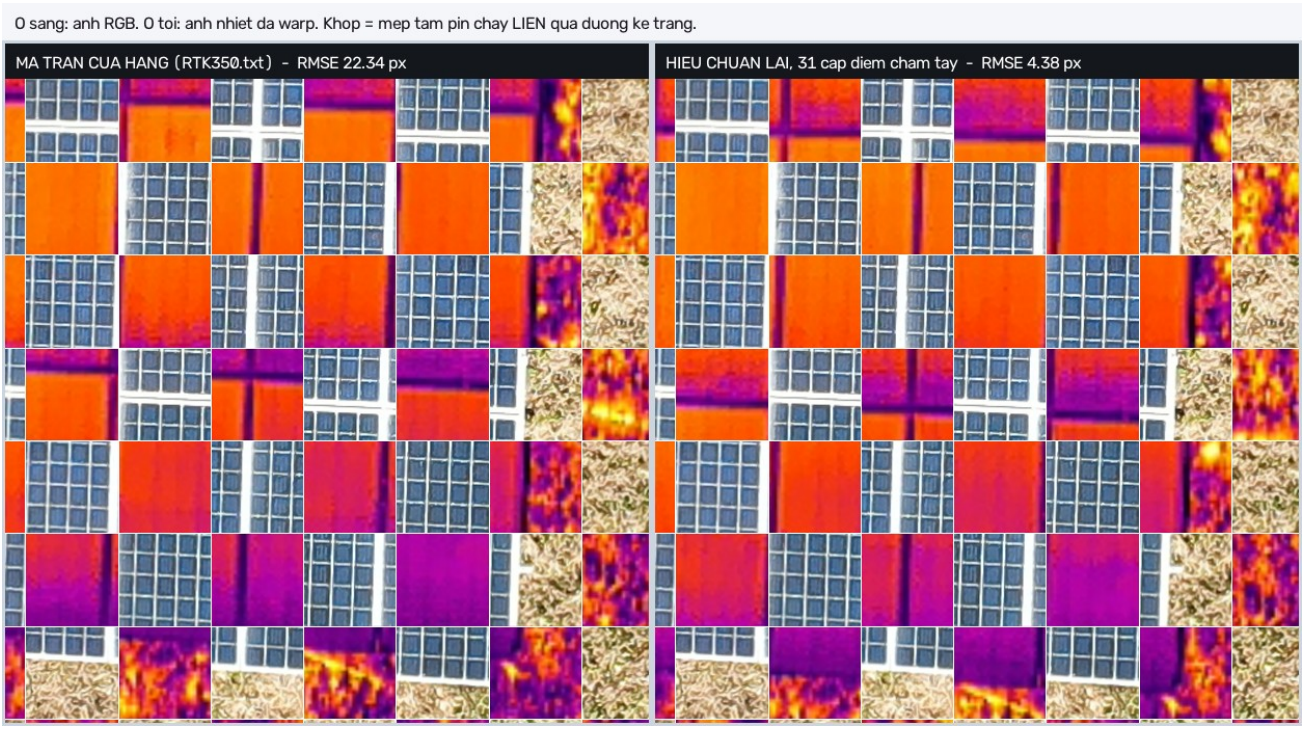
Hai điều dễ sai khi nối chuỗi:

- **H nhận toạ độ RGB đã khử méo ống kính** (`rgb_undistorted: true`). Đưa toạ độ RGB thô vào là lệch thêm ở rìa khung.
- **Phần bù trễ chụp `dx, dy` nằm ngoài H và đổi dấu theo chiều đi.** Chiều ngược (ortho → nhiệt) thì **cộng**; chiều xuôi (nhiệt → mặt đất) thì **trừ**, vì đang kéo pixel thật về lại hệ dự đoán mà H mô tả. Sai dấu thì lệch tâm tấm pin tăng từ 16 cm lên ~30 cm thay vì tụt xuống ~5 cm — ghi rõ ở đầu `quet_loi_anh_le.py`.

Sai số 4,35 cm ở trên vì thế **không đứng một mình**: nó là một hạng trong tổng sai số cuối, cộng với sai số tư thế camera, sai số DSM và sai số trễ chụp.

Bước 4 — so với ma trận của hãng

DJI có sẵn một ma trận trong `RTK350.txt`, ghi theo hệ **phần trăm khung ảnh** chứ không phải pixel. Đổi sang pixel bằng $\mathbf{H}_{px} = \mathbf{S}_{rgb} \cdot \mathbf{H}_{\%} \cdot \mathbf{S}_{nhit}^{-1}$ rồi so trực tiếp:



*Hình 4 — cùng cặp ảnh, cùng cách kiểm tra, chỉ khác ma trận. **Trái:** ma trận của hãng, RMSE 22,34 px — nhìn dải lối đi trắng ở ô RGB rồi bắt sang ô nhiệt kế bên, chúng lệch nhau thấy rõ. **Phải:** ma trận chấm tay, RMSE 4,38 px.*

> **Vì sao ma trận hãng lệch tới 22 px.** Nó là số hiệu chuẩn của dòng máy tại nhà xưởng, không phải của **con máy này**. Sai lệch lắp ráp từng chiếc cộng thêm việc gimbal đã tháo lắp làm nó trôi. Trong `thermal_to_rgb.json` còn một phần bù cố định nữa (`dx 9,65 px`, `dy 4,84 px`) suy từ hai ảnh bay đều `0320` và `0440` — đó là phần lệch lắp đặt không nằm trong H.

Vì sao đây là phần lõi tái sử dụng được

H gắn với **model drone**, không gắn với trang trại. Chấm một lần cho ZH20T là dùng được cho mọi trang trại bay bằng ZH20T. Tỉ lệ suy ra từ H cũng cho biết GSD ảnh nhiệt gốc **2,523 cm/px** — con số dùng lại ở khắp nơi về sau.

> **Đã thử tự động hoá và hỏng, đừng thử lại.** Tương quan pha thất bại vì lưới tấm pin tuần hoàn quá đều — đỉnh tương quan lặp lại ở mọi bước lưới, không có đỉnh nào trội. Ghép theo góc tấm + RANSAC thất bại vì detector tấm pin không chạy được trên ảnh nhiệt (conf tụt còn 0,30–0,38, đúng mức nhiễu). Chi tiết ghi ở đầu `ma/hieu_chuan_tu_dong.py`.

3.2 Lọc ảnh — bỏ ảnh xấu trước khi ghép

Ortho là ảnh ghép: một tấm lệch chỗ làm hỏng cả vùng quanh nó. Hai bộ lọc chạy trước, và chúng **chỉ phục vụ nhánh B**.

bộ lọc	cách đo	thời gian	nhận xét
Trễ màn trập	chiếu viền tấm pin qua H rồi trượt tìm chỗ khớp trong bán kính 16 px	~45 phút / 598 ảnh	bộ lọc tốt nhất
Rung thân máy	đọc EXIF	3 giây	nhanh nhưng thô — bắt trúng vài ảnh rất xấu, đồng thời quét nhầm nhiều ảnh tốt

Còn lại **396/598 ảnh** vào ortho nhiệt.

Hệ quả trực tiếp cho nhánh B. Vì ortho chỉ ghép từ 396 ảnh, có **14 trong 603 lỗi (2,3%)** chỉ nhìn thấy được từ những ảnh không vào ortho. Trên ortho chúng nằm giữa vùng đen — nhánh B không thể tìm ra chúng dù model tốt đến đâu.

> **Gợi ý đã kiểm chứng và bác bỏ:** lọc ảnh theo tốc độ bay trong EXIF. Đo trên cả 598 ảnh thì phân bố tốc độ của ảnh tốt và ảnh xấu chồng lên nhau, không tách được. Chốt ngưỡng trên mẫu 60 ảnh thì thấy "tách được" — đó là ảo giác của mẫu nhỏ.

3.3 Ghép ortho nhiệt — đi ngược chiều

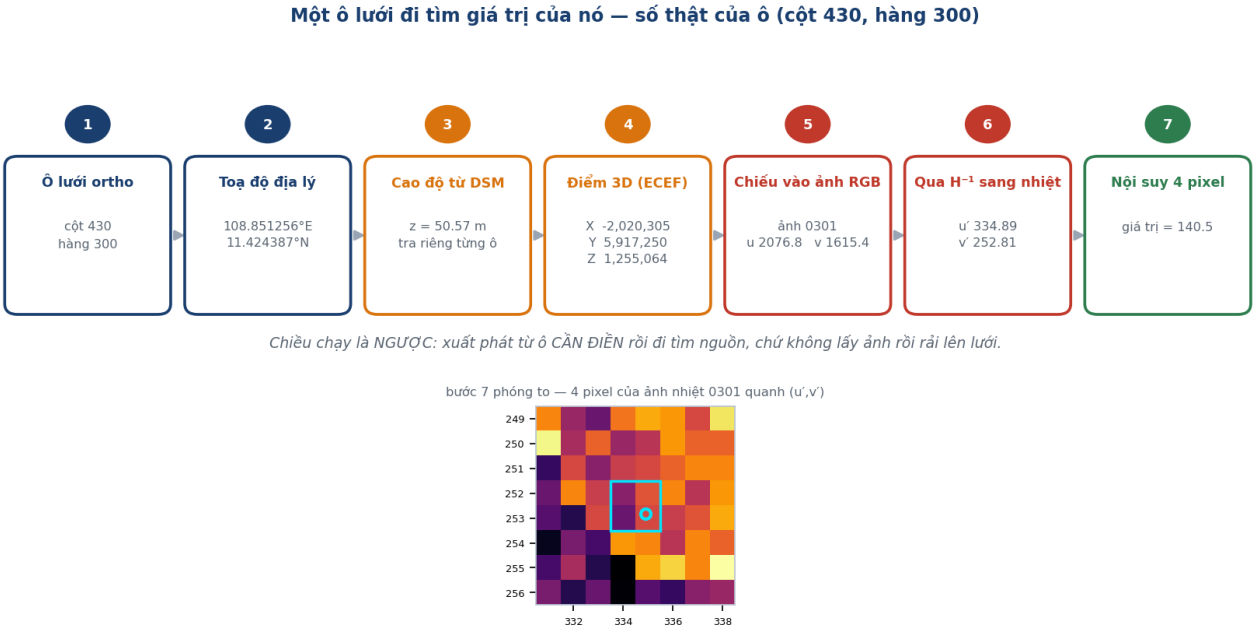
Ortho nhiệt là một tấm ảnh lớn phủ cả nhà máy, mỗi pixel ứng với một ô đất cố định. Câu hỏi là điền giá trị vào từng ô đó bằng cách nào. Có hai cách nghĩ ngược chiều nhau.

Cách thuận — "rải". Cầm từng ảnh nhiệt, tính xem mỗi pixel của nó rơi trúng ô nào trên lưới, rồi đặt xuống. Giống ném từng mảnh giấy lên bàn: chỗ không mảnh nào phủ tới thì mặt bàn trống — **thành lỗ**; chỗ hai mảnh chồng nhau thì phải quyết lấy mảnh nào — **thành mối nối**. Càng nhiều ảnh càng phải vá.

Cách ngược — "hỏi". Làm ngược lại: duyệt từng ô của lưới, mỗi ô tự hỏi *"tôi nằm ở chỗ nào ngoài đời, và ảnh nào chụp được chỗ đó?"* rồi đi lấy giá trị về. Không ai rải gì cả. Đây là cách dùng ở đây, và cũng là cách chuẩn trong đồ hoạ máy tính — *inverse mapping*.

Đổi chiều chạy giải quyết luôn cả hai vấn đề: mỗi ô được hỏi đúng một lần nên **không thể có lỗ**, và không bao giờ có chuyện hai ảnh tranh nhau một ô.

Bảy bước cho một ô lưới



*Hình 5 — bảy bước, với số thật chạy ra cho ô (cột 430, hàng 300). Bước 6 dùng **H nghịch** — chiều ngược với lúc chiếu box ở mục 5.1. Bước 7 nội suy song tuyến từ 4 pixel lân cận, ra giá trị 140,5.*

Đọc theo đúng ô trong hình — ô ở cột 430, hàng 300:

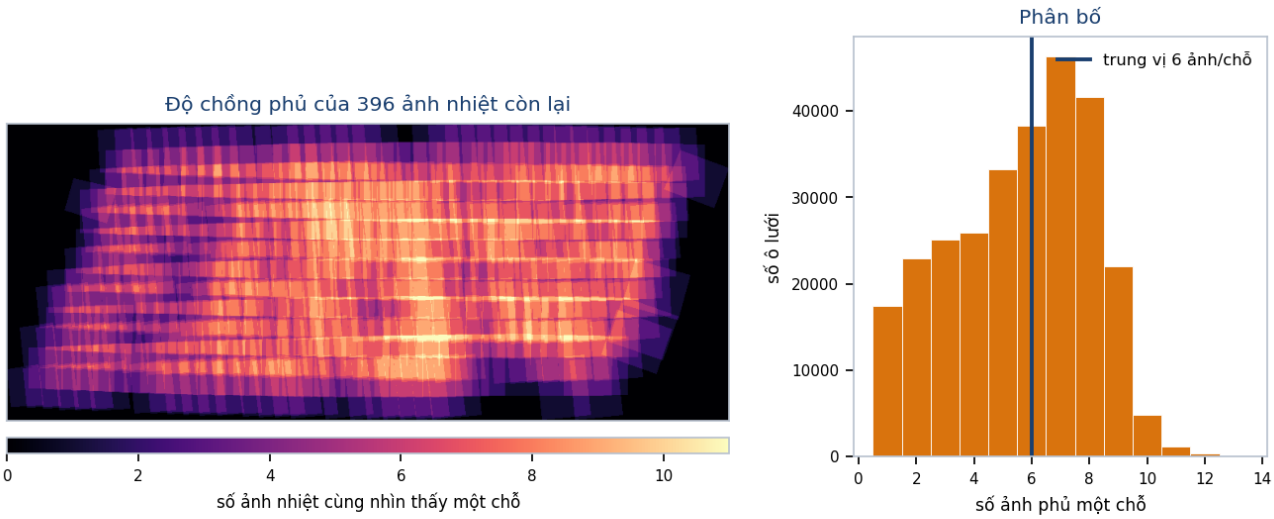
bước	câu hỏi ô đang tự hỏi	trả lời bằng cách nào	ra số gì
1	tôi là ô nào?	xuất phát từ một ô trống bất kỳ trên lưới	cột 430, hàng 300
2	tôi ở đâu trên bản đồ?	lưới ortho có toạ độ gắn sẵn (góc trên-trái + bước pixel), nên chỉ là một phép nhân-cộng	108,851256°E · 11,424387°N
3	chỗ đó cao bao nhiêu?	tra DSM — mô hình bề mặt do nhánh RGB dựng ra. Thiếu bước này thì không biết tia nhìn chạm vào đâu, cả ảnh trôi theo địa hình	z = 50,57 m
4	vậy tôi là điểm nào trong không gian?	gộp (kinh, vĩ, cao) thành một điểm 3D trong hệ ECEF — hệ toạ độ vuông góc lấy tâm Trái Đất làm gốc	X −2 020 305 · Y 5 917 250 · Z 1 255 064
5	ảnh nào nhìn thấy tôi?	cây KD tìm ảnh có tâm chụp gần nhất, rồi lấy tư thế + nội tham số của ảnh đó (Metashape đã giải) để chiếu điểm 3D vào khung ảnh	ảnh `0301`, px RGB (2076,8 · 1615,4)
6	chỗ đó trên ảnh **nhiệt** là pixel nào?	qua $\mathbf{H}^{-1}$ — đúng chiều ngược với lúc chiếu box ở nhánh A	px nhiệt (334,89 · 252,81)

bước	câu hỏi ô đang tự hỏi	trả lời bằng cách nào	ra số gì
7	giá trị bao nhiêu?	334,89 không phải số nguyên, nó rơi vào giữa 4 pixel; nội suy song tuyến theo khoảng cách tới từng pixel	**140,5**

Ba hệ quả của cách chạy này:

- **Không bao giờ có lỗ.** Ô nào cũng tự đi hỏi, hễ có ảnh phủ được là có giá trị. Vùng đen trên ortho là vùng **thật sự không ảnh nào nhìn thấy** — đó là thông tin về độ phủ, không phải khuyết tật của thuật toán.
- **Một ô lấy từ đúng một ảnh.** Ảnh gần nhất được hỏi trước; chỉ khi ảnh đó không phủ tới ô này thì mới hỏi sang ảnh gần nhì. Không trộn nhiều ảnh nên giá trị không bị pha loãng — đổi lại, ranh giới giữa hai vùng nguồn có thể lộ mối nối (Hình 7).
- **Chạy vài phút thay vì vài giờ.** Bảy bước trên là bảy phép toán trên mảng, làm một lượt cho cả ô vuông 2048×2048 bằng numpy, chứ không phải vòng lặp từng ô một.

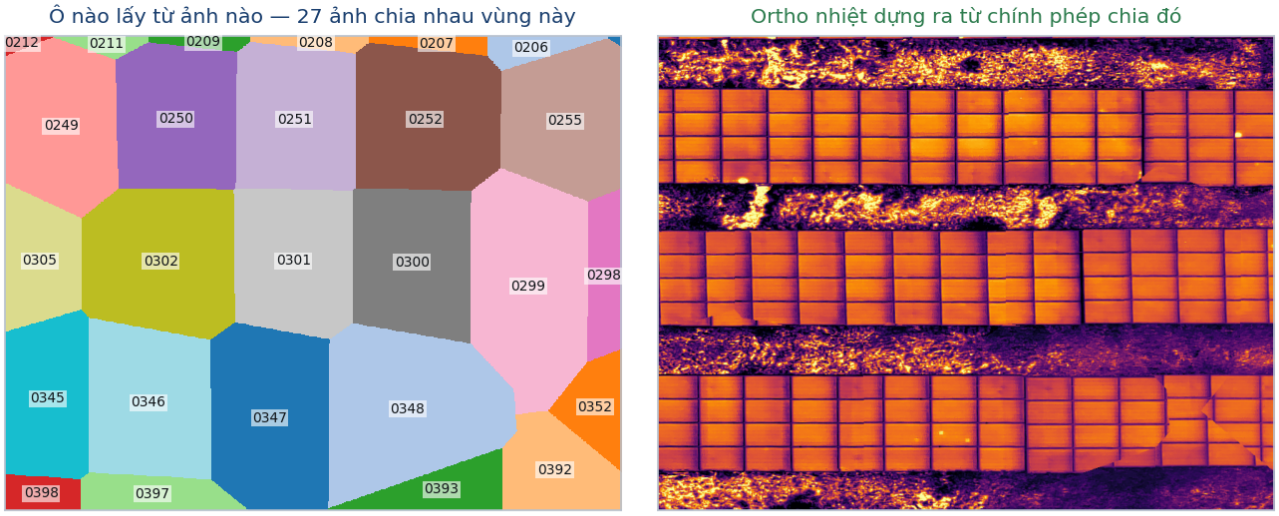
Mỗi điểm được bao nhiêu ảnh nhìn thấy



Trung vị 6 ảnh cùng nhìn thấy một chỗ. Chính vì dư nhiều như vậy nên bỏ 202 ảnh xấu vẫn không thủng ortho — và nên câu hỏi "lấy ảnh nào" mới cần trả lời có chủ đích.

Hình 6 — độ chồng phủ. Phần lõi nhà máy được 6–8 ảnh nhìn thấy, rìa thì ít hơn hẳn. Đây là gốc của trường n ở mục 5.2, và cũng là lý do ngưỡng $n \geq 3$ không mang sang chuyến bay cao hơn được.

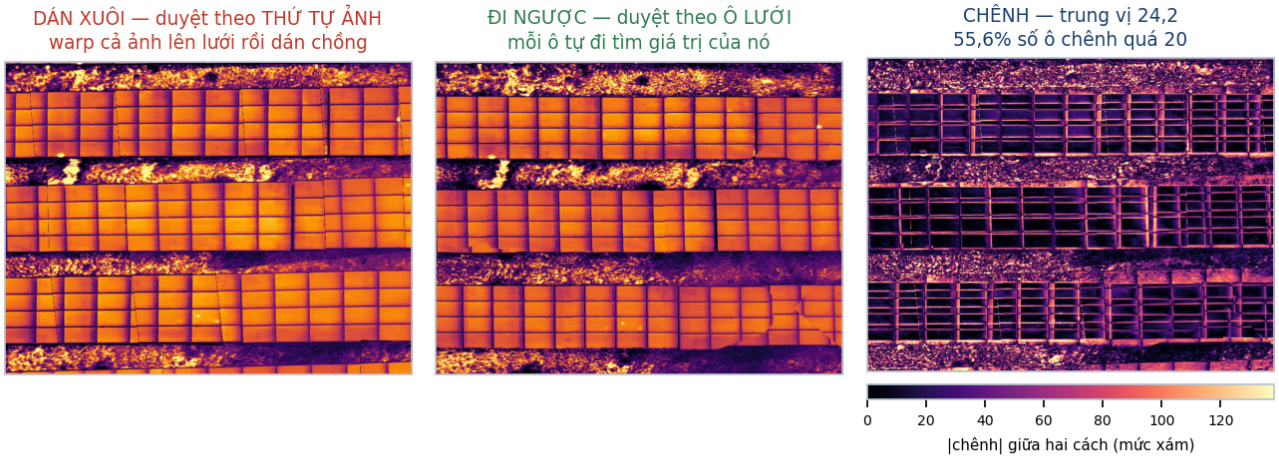
Ô nào lấy từ ảnh nào



Ranh giới là đường trung trực giữa hai tâm chụp — mỗi ô về tay ảnh có tâm gần nó nhất. Không có ảnh nào "dán sau đè lên ảnh trước": mỗi ô được điền đúng MỘT lần, và kết quả không phụ thuộc thứ tự duyệt ảnh.

*Hình 7 — **trái:** mỗi màu là một ảnh nguồn, nhãn là số thứ tự ảnh. **Phải:** ortho nhiệt dựng ra từ chính vùng đó. Ranh giới màu bên trái chính là chỗ mối ghép sẽ xuất hiện nếu hai ảnh nguồn lệch nhau.*

Xuôi và ngược cho ra hai kết quả khác nhau

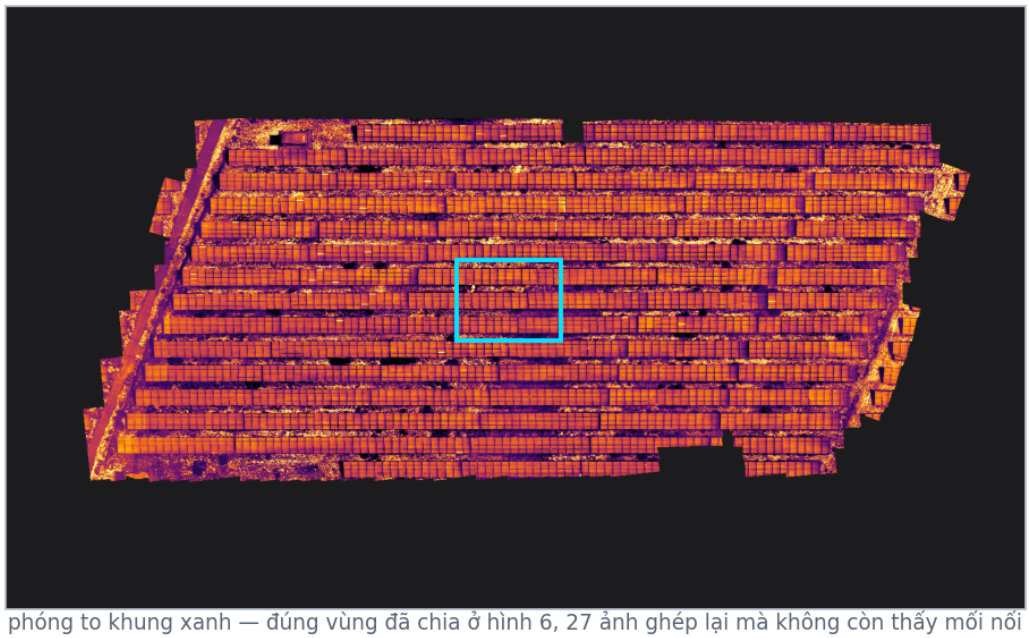


Cùng dữ liệu, cùng lưới, cùng 396 ảnh, cùng cân bằng sáng — khác duy nhất CHIỀU CHẠY. Dán xuôi ghi đè chồng lên nhau: ở vùng này mọi ô đều bị ghi lại nhiều lần, chỗ nhiều nhất 11 lần, và chỉ lần CUỐI được giữ — nên kết quả phụ thuộc thứ tự duyệt ảnh. Nó cũng phải giả định cả vùng ảnh phủ nằm trên một mặt phẳng (một phép biến đổi phối cảnh chỉ nối được hai mặt phẳng), trong khi đi ngược tra DSM cho từng ô nên bám được độ cao tấm pin — chênh nổi lên đúng ở mép các dãy pin.

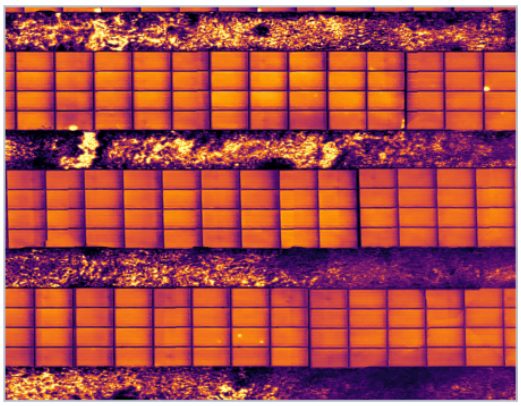
Hình 8 — cùng vùng, cùng lưới, cùng 396 ảnh, cùng cân bằng sáng. Khác duy nhất là chiều chạy.

Kết quả

ortho_nhiet_tu_lam.tif — 8451 × 4997 px, GSD 2,87 cm, ghép từ 396 ảnh nhiệt



phóng to khung xanh — đúng vùng đã chia ở hình 6, 27 ảnh ghép lại mà không còn thấy mối nối



Phần đen quanh nhà máy là khung chữ nhật của ortho màu, tuyến bay không phủ tới — đó là chỗ chênh giữa hai con số phủ ở hình 1.

*Hình 9 — ortho nhiệt toàn nhà máy **8 451 × 4 997** ở 2,871 cm/px, và phóng to đúng vùng đã chia ở Hình 7.*

Ba điều phải nhớ về ortho này

- Bước pixel **đúng gấp 3 lần** ortho màu, và cùng góc trên-trái. Hai lớp chồng khít nhau không cần nắn gì thêm — đó là lý do công cụ gán nhãn gạt qua lại giữa hai lớp được tại chỗ.
- Nhiều ảnh cùng phủ một pixel thì **không trộn**: lấy ảnh có tâm chụp gần nhất, các ảnh còn lại chỉ được hỏi tới khi ảnh đó không phủ được ô (`done[idx] = True`, `ghep_ortho_nhiet.py:394`). Bản ghép **có** trộn là một biến thể riêng — `ghep_ortho_pha_tron.py` — và chưa từng dựng ra file nào. Ghi rõ ở đây vì mục 7 lấy "blend làm nhoè hotspot" làm nghi can chính, trong khi ortho đang dùng vốn không blend.
- Nguồn đọc bằng `IMREAD_GRAYSCALE`, nên ortho **mất bảng màu ironbow** mà model đã train trên đó. Đây là biến phải kiểm soát riêng ở mục 7.1, nếu không nó lẫn vào phần mất do ghép.

  Cân bằng sáng không xoá được mối ghép. Đo trên 3 cửa sổ: chênh lệch mức xám giữa hai dải bay chỉ giảm từ **18,1 xuống 17,7**. Đứt đoạn nhìn thấy ở mép là **hình học** — sai số đặt chỗ giữa các ảnh

— chứ không phải chênh độ sáng. Chỉnh gain không cứu được.

---

4. Model phát hiện lỗi

`fault_obb_4class.pt` — YOLO-OBB. Hộp **xoay** chứ không phải hộp thẳng trục, vì tấm pin nằm nghiêng theo hướng bay và một hộp thẳng trục bao quanh tấm nghiêng thì thừa gần một nửa diện tích.

lớp	số ca (nhánh A)	ý nghĩa
`SingleHotSpot`	429	điểm nóng đơn lẻ
`MultiHotSpot`	126	cụm điểm nóng
`Bypass`	48	diode bypass đang dẫn
`ModuleOpenCircuit`	0	không gặp trên bộ Thuận Nam

Box của model bao gần trọn một tấm pin. Trung vị **0,95 × 1,85 m** so với tấm pin 0,88 × 1,76 m. Tức model vốn đã phán ở **mức tấm**, không khoanh riêng ô cell nóng. Đừng đọc dấu tâm box như vị trí ô lỗi.

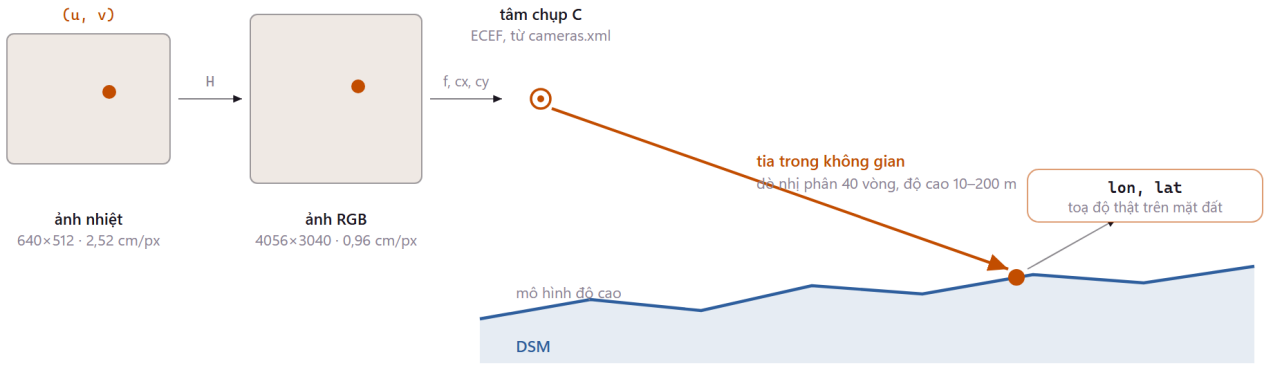
Ngưỡng chốt: conf 0,25. Đẩy lên 0,64 (tối ưu F1) thì mất **146/603** lỗi mà chất lượng chỉ nhích 2 điểm; trong 146 cái mất đi có **129 cái rơi đúng trong tấm pin**, tức cứ dọn 1 box rác thì vứt nhầm 7,6 lỗi thật.

---

5. Nhánh A — detect trên ảnh lẻ rồi chiếu

Đây là nhánh đang chạy trong pipeline.

5.1 Chuỗi chiếu: từ một pixel tới một điểm trên mặt đất



Hình 10 — chuỗi chiếu, chạy 4 lần cho mỗi box (một lần cho mỗi góc).

Các bước, theo đúng thứ tự trong `core/ortho/projection.py`:

1. **Pixel ảnh nhiệt (u, v)** — tâm hoặc góc của box YOLO.
2. **Qua H** sang pixel ảnh RGB tương ứng.
3. **Khử méo + nội tham số** (`f`, `cx`, `cy` từ `cameras.xml`) → vector chỉ phương của tia trong hệ camera. Lưu ý quy ước Metashape: `cx`, `cy` là **độ lệch so với tâm ảnh**, không phải toạ độ tuyệt đối.
4. **Đổi sang hệ ECEF** bằng ma trận tư thế của đúng ảnh đó cộng phép biến đổi chunk.
5. **Cắt DSM** bằng dò nhị phân 40 vòng dọc tia, quét độ cao 10–200 m.
6. **Giao điểm → lon/lat** qua `pyproj`.

Hình 10 vẽ **một điểm** đi hết chuỗi. Hai chỗ khó hình dung nhất lại không nằm ở đó, nên có thêm hai hình dưới đây.

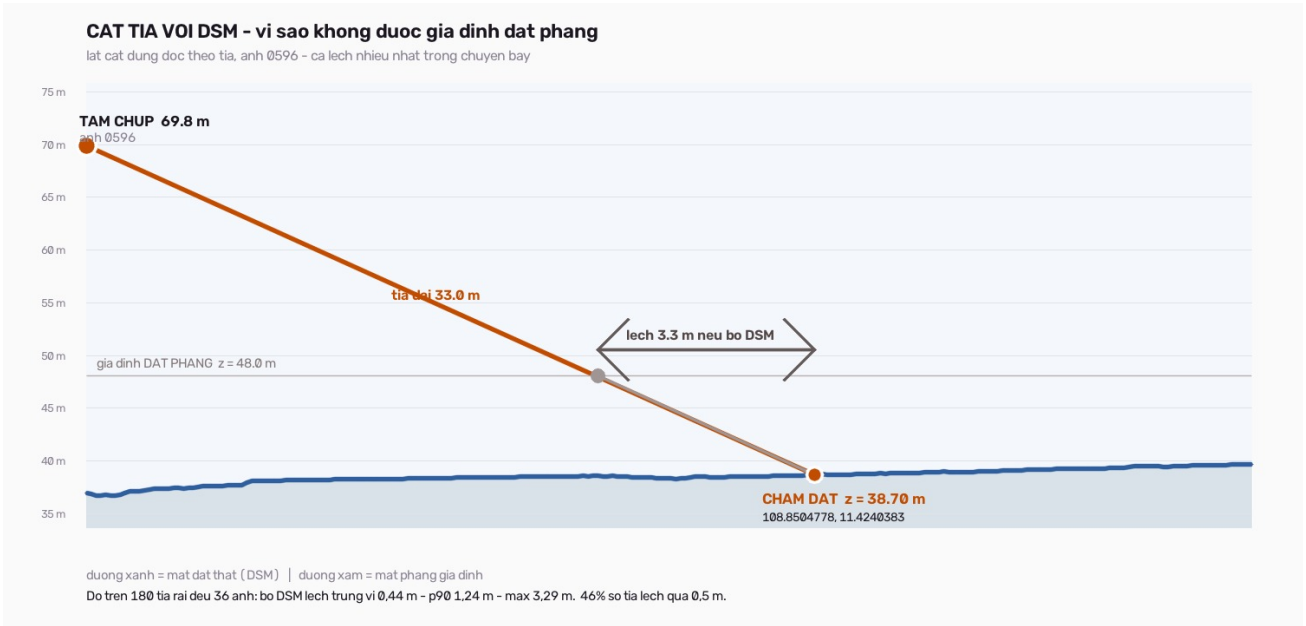
Vì sao là "4 góc" chứ không phải một điểm



Hình 10b — cùng một lỗi (`0005`, SingleHotSpot, conf 0,99) đi qua ba hệ toạ độ. Bốn góc đánh số 1–4 giữ nguyên thứ tự ở cả ba khung, toạ độ thật ghi ở dải dưới. ***Khung 1:*** *box trên ảnh nhiệt 640×512.* ***Khung 2:*** *4 góc sau khi qua H, giờ là pixel trên ảnh RGB 4056×3040.* ***Khung 3:*** *sau khi phóng tia và cắt DSM, bốn góc thành bốn toạ độ GPS thật trên ortho màu.*

Chiếu cả bốn góc chứ không chỉ tâm, vì box chụp xiên đổ xuống mặt đất thành **hình bình hành** chứ không còn là chữ nhật. Đó là vết thật của lỗi trên mặt đất, và chính nó — không phải `w_m`/`h_m` — được dùng để tính IoU với polygon tấm pin.

Vì sao phải cắt DSM



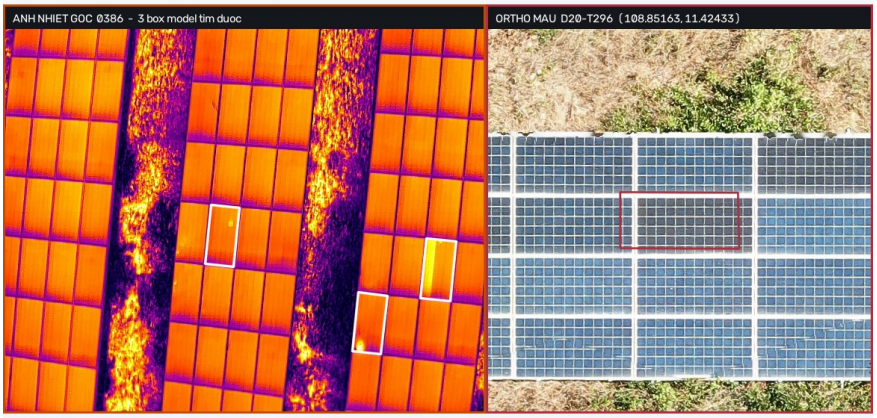
Hình 10c — ca lệch nhiều nhất trong chuyến bay (ảnh 0596). Mặt đất thật ở ***38,70 m*** *nhưng mặt phẳng giả định ở* ***48,0 m*** *— tia dừng sớm 9 m theo phương đứng, điểm chạm trượt* ***3,3 m*** *ngang.*

Đo trên 180 tia rải đều 36 ảnh: bỏ DSM thì lệch **trung vị 0,44 m, p90 1,24 m, tối đa 3,29 m**, và **46% số tia lệch quá 0,5 m**. Tấm pin rộng ~1 m, nên mức đó đủ để gán nhầm lỗi sang tấm bên cạnh. Địa hình nhà máy chênh 23,85 m giữa p1 và p99 — không phải mặt phẳng.

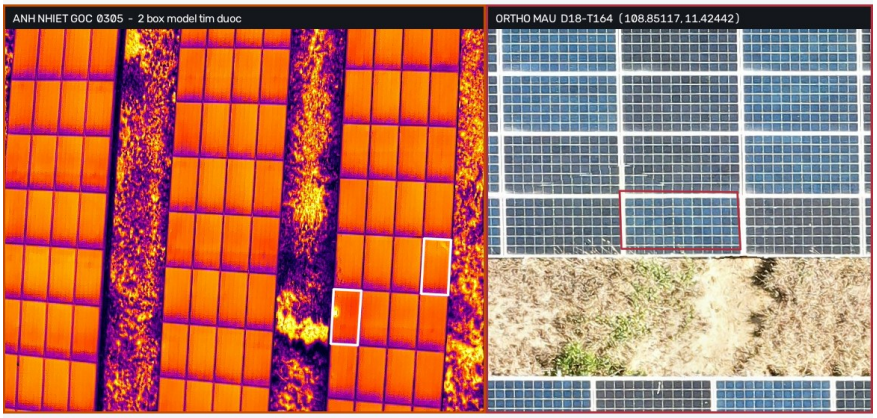
> **Đừng suy từ một ca.** Chính ca `0005` dùng ở Hình 10b nằm đúng chỗ đất phẳng, bỏ DSM chỉ lệch **5 cm**. Nhìn một tia rồi kết luận "DSM không cần thiết" là bẫy — phải nhìn phân bố.

Bốn ca thật của chính chuyến bay này:

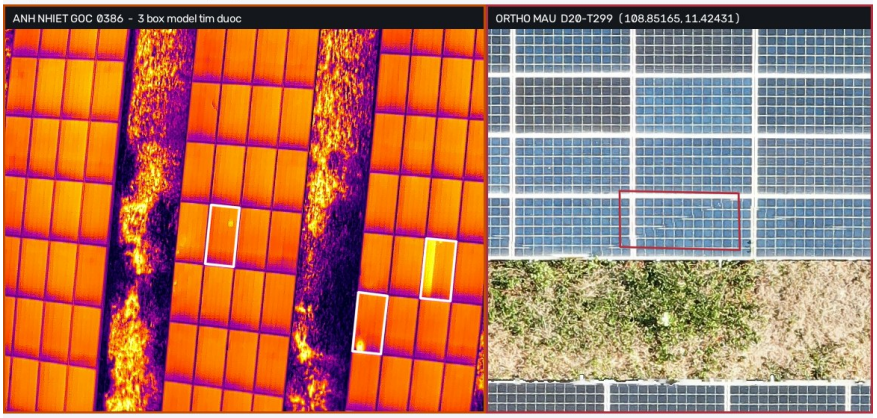
Bypass conf 0.99 6 anh cung thay IoU voi tam pin 0.66



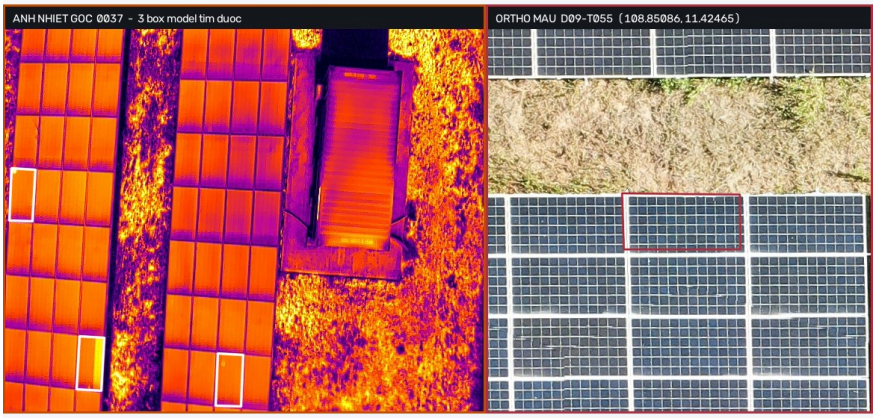
SingleHotSpot conf 0.98 6 anh cung thay IoU voi tam pin 0.80



SingleHotSpot conf 0.98 5 anh cung thay IoU voi tam pin 0.66



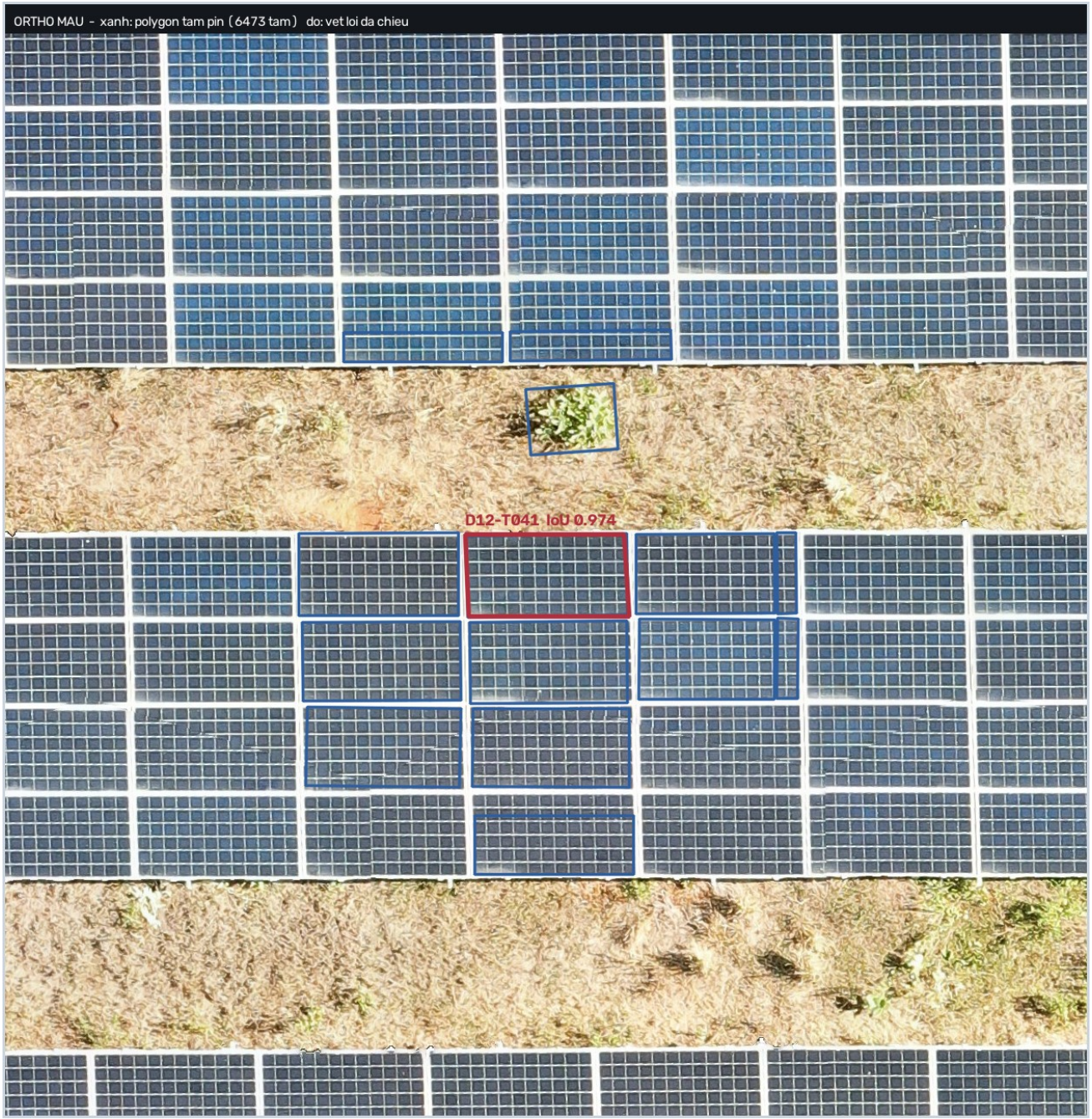
Bypass conf 0.98 8 anh cung thay IoU voi tam pin 0.85



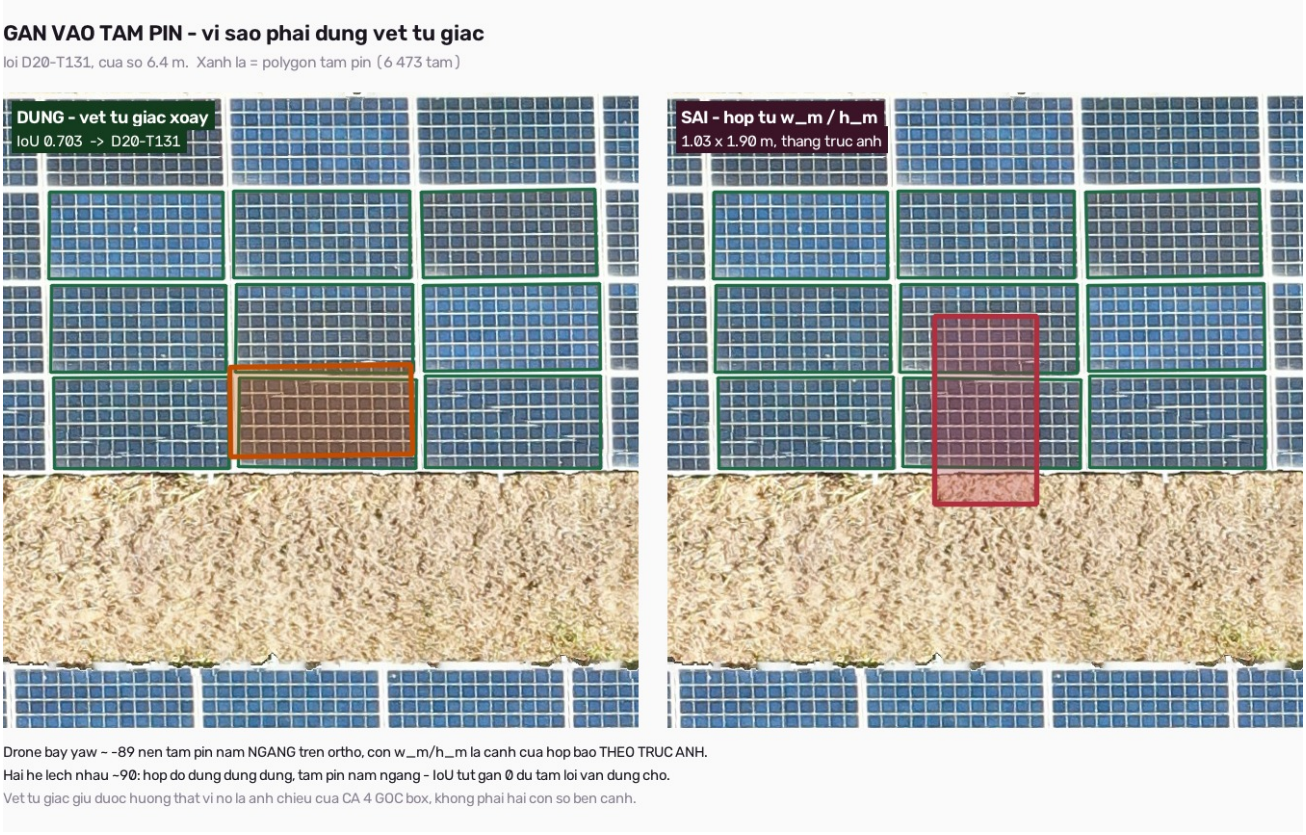
*Hình 11 — chuỗi chiếu chạy trên dữ liệu thật. **Trái:** ảnh nhiệt gốc 640×512, khung trắng là mọi box YOLO tìm được trên ảnh đó. **Phải:** ortho màu cắt cửa sổ 6 m quanh lỗi, khung đỏ là vết của lỗi ghi ở tiêu đề sau khi chiếu qua DSM. Hình không tuyên bố cặp box nào ứng với vết nào — cái cần thấy là box trên ảnh gốc và vết trên mặt đất cùng rơi vào một tấm pin. Chú ý ca cuối: vết đỏ méo thành hình bình hành, đó là hiệu ứng chụp xiên.*

5.3 Gán vào polygon tấm pin

Bước cuối, **dùng chung cho cả hai nhánh**. Có 6 473 polygon tấm pin dò sẵn trên ortho màu. Mỗi lỗi được gán bằng hai phép cùng lúc: điểm-trong-đa-giác cho tâm lỗi, và IoU giữa **vết tứ giác xoay** với polygon tấm.



*Hình 12 — gán tấm pin, cửa sổ 12 m. **Xanh:** polygon tấm pin. **Đỏ:** vết lỗi đã chiếu, IoU 0,974 với tấm `D12-T041`.*



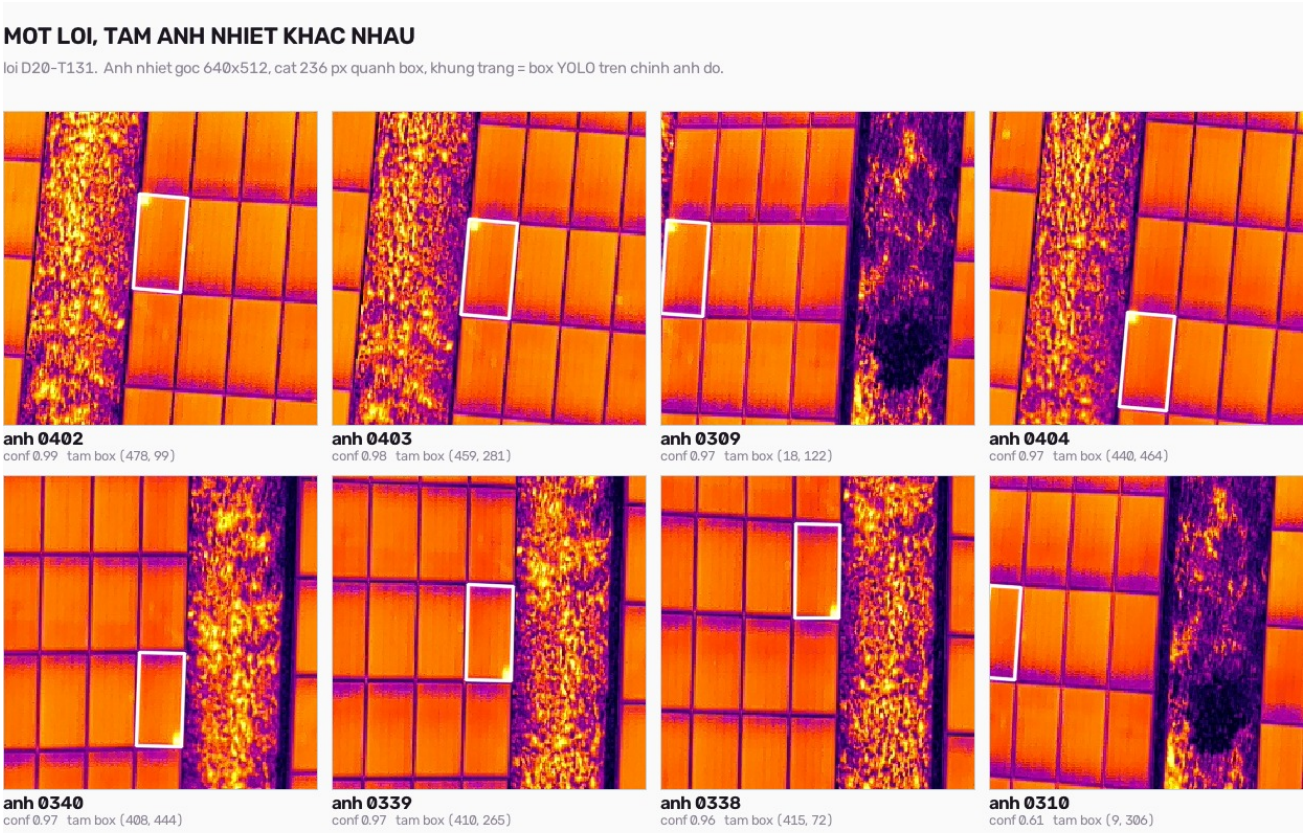
Hình 12b — vì sao phải dùng vết tứ giác. Cùng lỗi `D20-T131`, cùng vùng, xanh lá là polygon tấm pin. ***Trái:*** *vết tứ giác xoay — ảnh chiếu của cả 4 góc box — nằm khít một tấm, IoU 0,703.* ***Phải:*** *hộp dựng từ `w_m/h_m` (1,03 × 1,90 m) đứng vuông góc, cắt ngang ba tấm.*

Phải dùng vết tứ giác, không được dùng `w_m/h_m`. Hai số đó là cạnh của **hộp bao theo trục ảnh**, mà drone bay yaw ≈ −89° nên chúng lệch ~90° so với hướng nằm của tấm pin: box đo ra 0,95 × 1,85 m trong khi tấm pin trên ortho là 1,80 × 1,05 m. Dựng hộp IoU từ hai số đó là vuông góc với tấm pin, IoU ra gần 0. Hình 12b là chính ca đó: tâm lỗi vẫn đúng chỗ, chỉ hướng là sai, mà thế đã đủ hỏng phép gán.

Polygon tấm pin không hoàn hảo. Trong Hình 12 có một khung xanh nằm giữa bãi cỏ — detector tấm pin bắt nhầm một mảng cỏ. Đó là lý do bước gán luôn báo kèm `in_panel` và `dist_cm`, để phân biệt "lỗi nằm ngoài mọi tấm" với "lỗi nằm trong một tấm ma".

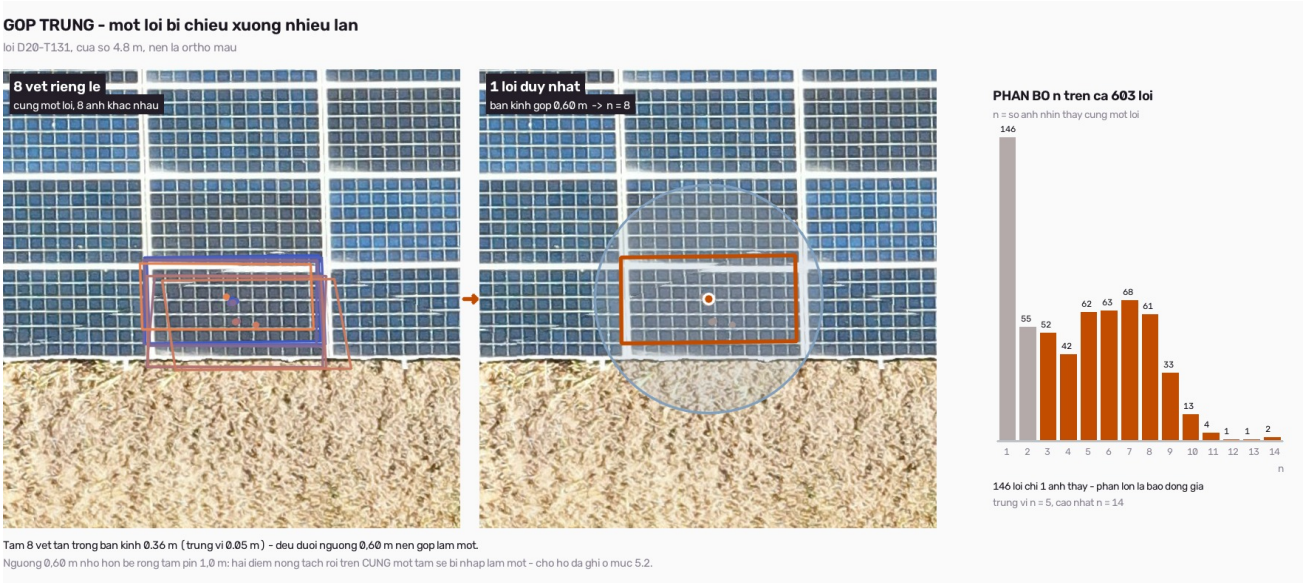
5.2 Gộp trùng, và trường `n`

Chuyến bay chồng phủ cao nên một tấm pin nằm trong nhiều ảnh. Detect trên ảnh lẻ nghĩa là cùng một lỗi bị bắt nhiều lần — phải gộp. Nhưng **số lần bị bắt tự nó là thông tin**.



*Hình 12c — **chồng lấn là chuyện giữa các ảnh nhiệt với nhau**, không phải giữa nhiệt và RGB. Tám ảnh nhiệt gốc cùng bắt được lỗi `D20-T131`, khung trắng là box YOLO trên chính ảnh đó. Tâm box chạy từ (18, 122) ở ảnh `0309` tới (478, 99) ở ảnh `0402` — cùng một tấm pin nhưng rơi vào chỗ khác nhau trong khung, vì drone đứng ở tám vị trí khác nhau lúc bấm. Ba đường bay: `0309–0310`, `0338–0340`, `0402–0404`.*

Quan hệ nhiệt↔RGB là **1–1 theo cặp**: mỗi ảnh `_T` có đúng một ảnh `_W` chụp cùng lúc, ghép theo số seq. Tám lần chiếu ở trên là tám lần chạy độc lập, mỗi lần dùng ảnh RGB cùng số hiệu. H là **cùng một ma trận** cho cả tám; thứ khác nhau là **tư thế camera**.



*Hình 13 — gộp trùng trên lỗi thật `D20-T131`. **Trái:** 8 vết do 8 ảnh khác nhau chiếu xuống, tâm tản trong bán kính 0,36 m (trung vị 0,05 m). **Giữa:** gộp bán kính 0,60 m thành một lỗi, `n = 8`. **Phải:** phân bố n trên cả 603 lỗi — 146 lỗi chỉ một ảnh thấy, và đó phần lớn là báo động giả.*

Chỗ này cho thấy vì sao chuỗi chiếu **không dừng ở toạ độ GPS**. Chiếu xong mới chỉ có 2 756 vết rời rạc; chúng phải được nhận ra là "cùng một lỗi" thì mới đếm được có bao nhiêu lỗi thật.

Chồng lấn thế thì lấy ảnh nào? Câu này là chỗ hai nhánh tách hẳn nhau.

	nhánh A — ảnh lẻ	nhánh B — ortho
bắt buộc chọn một ảnh?	**không**	**có** — mỗi ô lưới chỉ điền được một giá trị
chọn thế nào	giữ cả 8, gộp theo vị trí	ảnh có tâm chụp gần nhất (mục 3.3)
toạ độ lỗi lấy từ đâu	vết **conf cao nhất** làm hạt nhân, 7 vết còn lại chỉ cộng vào n	không có khái niệm này
chồng lấn là gì	**dữ liệu** — sinh ra trường n	**vấn đề** — phải chọn, sinh ra mối ghép

Nhánh A sắp các vết theo conf giảm dần rồi lấy vết đầu tiên làm hạt nhân của cụm (`quet_loi_anh_le.py:243`) — **không lấy trung bình 8 vết**. Đó là một chỗ hở đã ghi ở cuối mục 5.2: trung bình n điểm sẽ giảm sai số khoảng $1/\sqrt{n}$, hiện đang bỏ phí.

Quan sát thô	**2 756** box trên 598 ảnh lẻ
Sau gộp bán kính 0,60 m	**603** lỗi riêng biệt
Số ảnh cùng thấy một lỗi (n)	từ 1 đến 14, trung vị 6
n của nhóm đúng (TP)	trung vị **6**
n của nhóm báo động giả (FP)	trung vị **1** — 64% FP chỉ hiện đúng một ảnh

Trường n lọc tốt hơn conf. Ở cùng recall 0,831 và cùng 368 TP:

lọc	FP	Precision
`n ≥ 3`	**12**	**0,968**
`conf ≥ 0,64`	19	0,951

> **Ngưỡng `n ≥ 3` không mang đi được.** Nó gắn với độ chồng phủ ~7 ảnh/điểm của chuyến bay 31 m. Bay cao hơn thì chồng phủ tụt (ước ~3 ở 85 m) nên ngưỡng phải hiệu chỉnh lại.

Ba chỗ hở trong khâu gộp, chưa sửa: lớp lấy theo box conf cao nhất thay vì bầu theo cả cụm; toạ độ lấy từ một box thay vì trung bình n điểm — mất cơ hội giảm sai số khoảng 1/√n; và bán kính 0,60 m nhỏ hơn bề rộng tấm 1,0 m nên hai điểm nóng tách rời trên cùng một tấm bị nhập làm một.

---

6. Nhánh B — quét thẳng trên ortho

Không cần chiếu, không cần DSM, không cần gộp giữa các ảnh. Ortho đã là lưới địa lý — mỗi pixel có sẵn toạ độ, đọc thẳng ra lon/lat.

Cách quét	ô **640×512** (đúng kích thước ảnh model quen), trượt với chồng lấn 20%
Số ô có dữ liệu	103 ô trên toàn nhà máy

Kết quả	758 box thô → gộp 0,60 m → **463 lỗi**
Trường n	không có — mỗi lỗi chỉ tồn tại một lần trên ortho

Nhánh này thắng rõ ở khâu định vị, vì nó không có sai số chiếu:

	nhánh A (ảnh lẻ)	nhánh B (ortho)
Rơi đúng trong polygon tấm pin	90,2%	**91,6%**
Gán được mã tấm	93,2%	**93,5%**
IoU vết lỗi ↔ tấm pin (trung vị)	0,716	**0,891**
Lệch tâm (trung vị)	16 cm	**5 cm**

Hợp lý: nhánh A phải đi qua H, tia, DSM — mỗi chặng góp một ít sai số — còn nhánh B đọc thẳng toạ độ có sẵn.

---

7. Phép so hai nhánh

7.1 Bốn điều kiện

Muốn biết ghép ortho làm mất bao nhiêu thì phải **giữ nguyên model và chỉ đổi thứ đưa vào nó**. 603 lỗi mốc, conf 0,25, bắt trúng = trong 1 m.

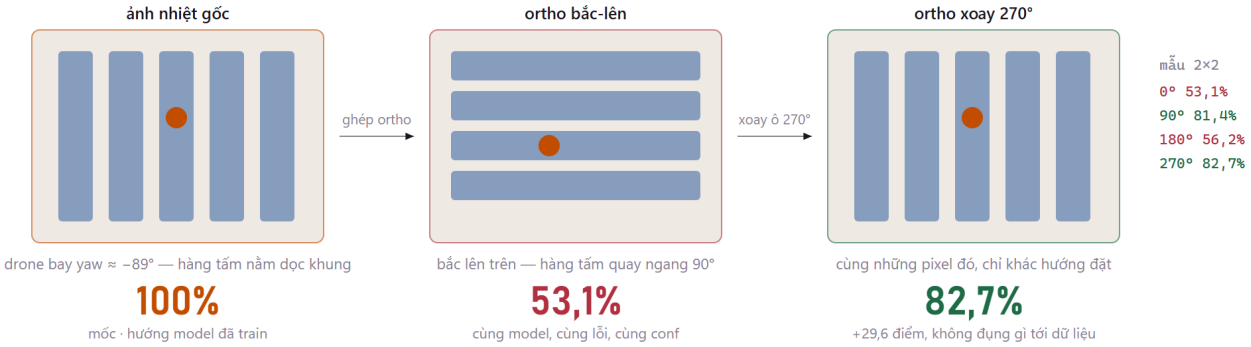
	Đầu vào đưa vào model	Giữ lại được	Đo được gì
A	ảnh lẻ + ironbow gốc	100%	mốc theo định nghĩa — 603 lỗi do chính nó sinh ra
B	ảnh lẻ + xám→INFERNO	**89,6%**	phần mất riêng do **đổi bảng màu**
C	ortho + INFERNO, **bắc-lên**	**53,1%**	tưởng là "ghép làm nhoè hotspot"
C′	ortho + INFERNO, **xoay 270°**	**82,7%**	cùng ortho đó, chỉ đổi hướng đặt ô

Tách nguyên nhân:

nguyên nhân	mất
đổi bảng màu ironbow → INFERNO	−10,4 điểm
hướng cảnh	**−29,6 điểm** (lớn nhất)
ghép ortho thật sự (blend + lấy mẫu lại)	−6,9 điểm

> **Điều kiện B là chỗ dễ bỏ sót nhất.** Nếu chỉ so A với C thì được con số 53% và gán hết cho "ghép ortho" — trong khi hơn một phần năm của phần mất đó chỉ là do ortho không giữ được bảng màu ironbow. Bản đầu của `ma/thu_detect_tren_ortho.py` thiếu đúng điều kiện này.

7.2 Thủ phạm là hướng cảnh



Hình 13 — cùng một model, cùng những pixel đó, chỉ khác hướng đặt ô ảnh.

Drone bay yaw ≈ **−89°** nên trên ảnh gốc hàng tấm pin nằm dọc khung. Ortho là bắc-lên, nên cảnh **xoay ~90°** so với mọi ảnh model từng thấy lúc train. Cùng gốc với cái bẫy `w_m/h_m` lệch 90° gặp khi viết `gan_tam_pin_4lop.py`.

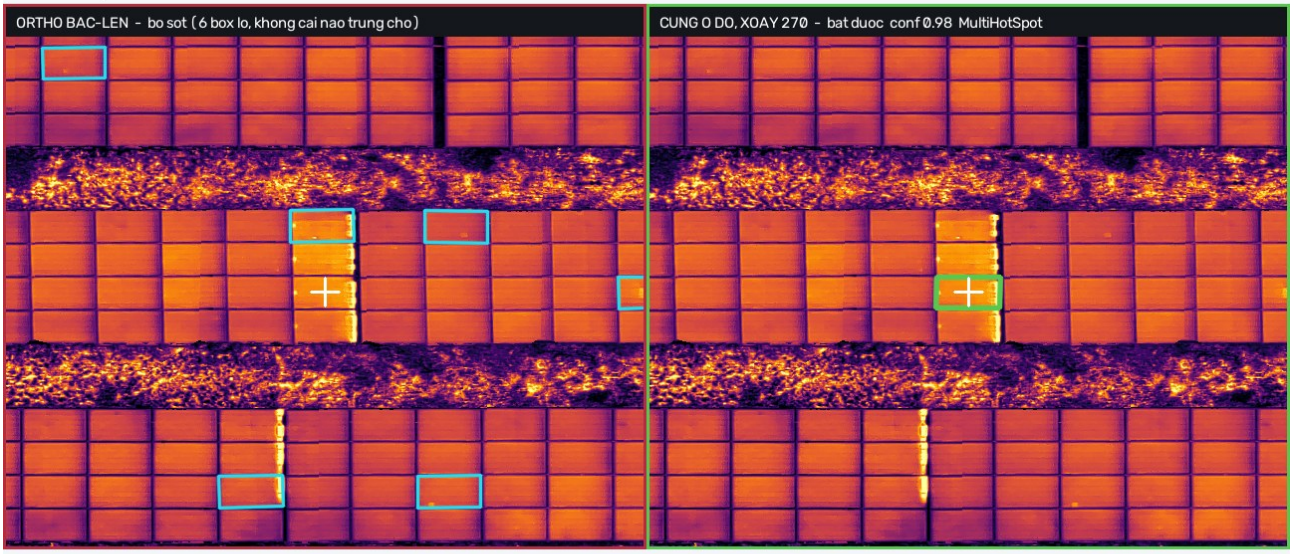
Mẫu 2×2 là bằng chứng chốt:

xoay ô	0°	90°	180°	270°
recall	53,1%	**81,4%**	56,2%	**82,7%**

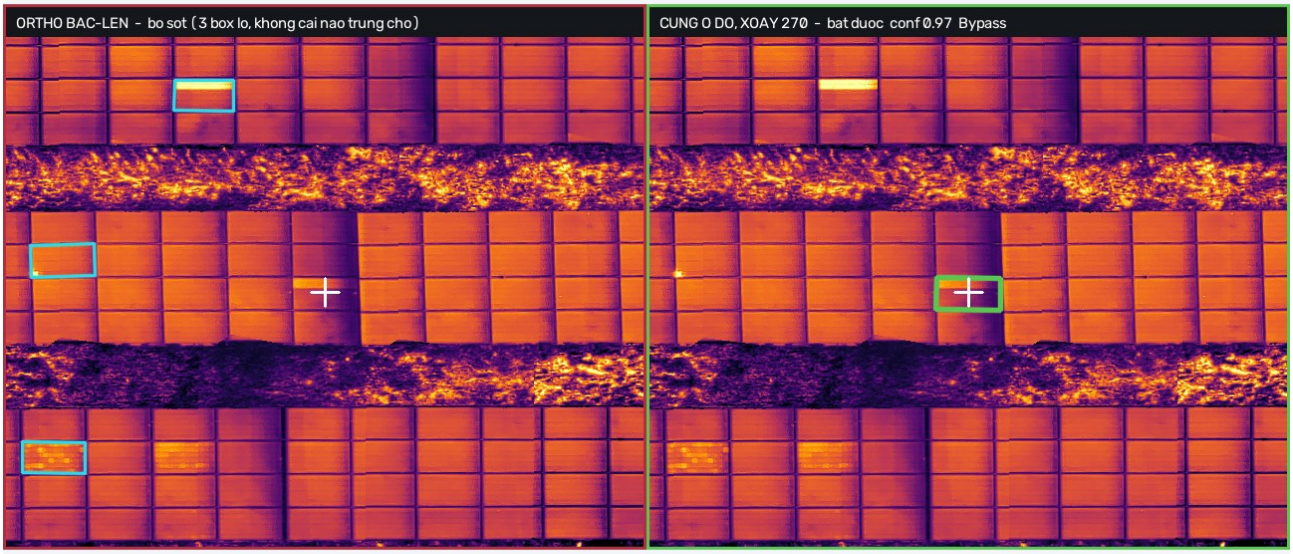
0° và 180° cùng kém, 90° và 270° cùng tốt — vì xoay 180° giữ nguyên hướng hàng tấm. Nếu nguyên nhân là ortho làm mất tín hiệu thì xoay ảnh không thể cứu được gì.

Ba ca thật, nhìn là hiểu ngay:

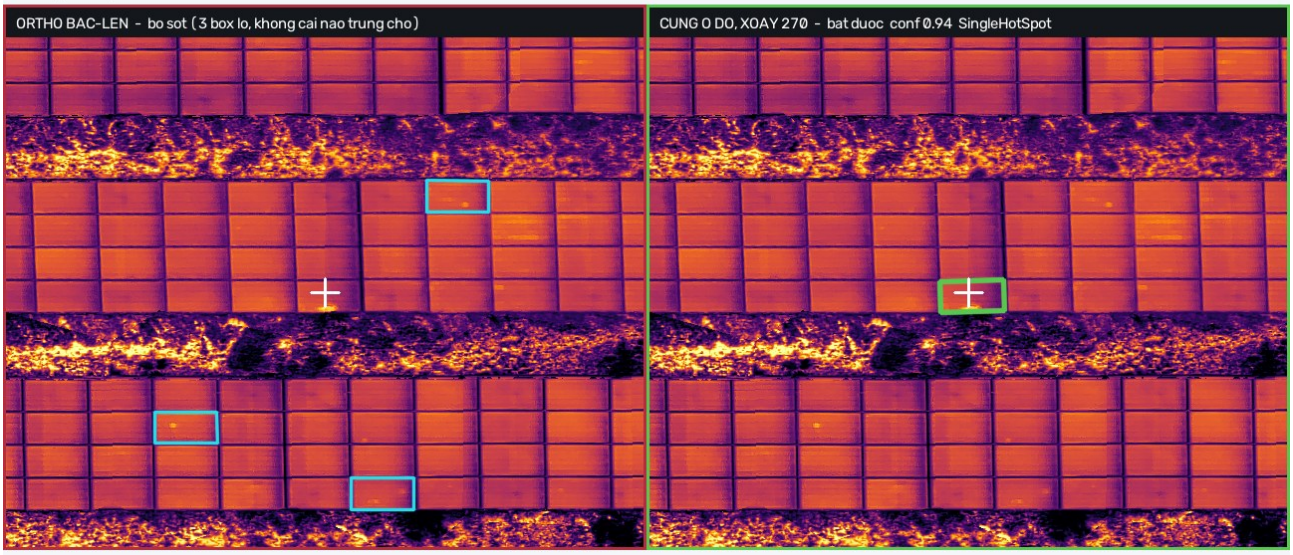
loi MultiHotSpot tai tam o - D11-T272



loi Bypass tai tam o - D11-T188



loi SingleHotSpot tai tam o - D32-T037



Hình 14 — cùng một ô ortho, đưa vào model hai lần. ***Trái, bắc-lên:*** *model bỏ sót — chữ thập trắng là lỗi thật, khung lơ là những box model vẫn nổ ra nhưng ở chỗ khác.* ***Phải, xoay 270°:*** *cùng những pixel đó, model bắt trúng với conf 0,94–0,98. Ảnh bên phải đã quay về bắc-lên sau khi vẽ box, để hai bên so được trực tiếp.*

Điểm đáng chú ý ở cột trái: model **không hề im lặng** — nó vẫn nổ ra 3–6 box trên mỗi ô, chỉ là không cái nào rơi vào chỗ có lỗi. Đó là hành vi của một model đang nhìn cảnh lạ, không phải của một model đang nhìn ảnh mất tín hiệu.

7.3 Kiểm chứng ngược: nét không phải là tương phản

Giả thiết ban đầu — ghi trong docstring của `quet_loi_anh_le.py` và trong `HUONG_DAN_PIPELINE.md` — là *"ortho nhiệt là ảnh blend trung bình nhiều ảnh chồng phủ, hotspot nhỏ bị làm nhoè"*. Đo trực tiếp hai đại lượng trên cùng những vị trí lỗi, cùng đọc từ ảnh xám, cùng bán kính vật lý:

đại lượng	ảnh lẻ	ortho
Độ nét cấu trúc — var(Laplacian), cái mắt gọi là "nét"	14 362	**14 801**
Tương phản nhiệt — đỉnh vệt nóng trừ nền quanh nó	46,0	42,0

Hai đại lượng suy biến theo hai kiểu khác nhau. **Viền tấm pin** là cấu trúc lớn, tương phản mạnh, nằm đúng chỗ trong mọi ảnh chồng phủ — trung bình hoá không xoá được; ortho thậm chí nét hơn ảnh gốc vì nó lấy mẫu lại từ nhiều nguồn. **Vệt nóng** thì nhỏ, vị trí lệch vài pixel giữa các ảnh, nên trung bình hoá kéo đỉnh xuống — nhưng chỉ kéo xuống **9%**, không đủ để giải thích mất 47%.

> **Manh mối đã chỉ ngay từ đầu.** Nhóm lỗi bị bỏ sót có tương phản 37,2 còn nhóm bắt được là 46,0 — nhưng hai phân bố **chồng lấn nặng**: p75 của nhóm bỏ sót là 49,0, cao hơn cả trung vị nhóm bắt được. Nếu nhoè là thủ phạm thì hai phân bố phải tách rời. Chính chỗ chồng lấn đó dẫn tới giả thuyết hướng cảnh.

---

8. Đánh giá bằng nhãn tay

511 tấm gán tay trên ortho (443 có lỗi, 68 tốt). Đơn vị chấm là **tấm pin** chứ không phải box — người vận hành nhận danh sách tấm để đi thay, không ai đi thay "một box", và model bắn nhiều box vào cùng một tấm là chuyện thường.

nhánh	TP	FP	FN	Precision	Recall	F1
A · ảnh lẻ (đang dùng)	429	67	14	0,865	**0,968**	0,914
B · ortho xoay 270°	381	15	62	**0,962**	0,860	0,908
A + lọc n ≥ 3	368	12	—	0,968	0,831	—

Nhánh ortho đổi recall lấy precision, và đổi rất được giá: F1 gần như hoà. Lý do có lẽ chính là cái blend từng bị đổ tội — nó xoá vệt nóng yếu, nhưng cũng **xoá luôn báo động giả thoáng qua**. 64% FP của nhánh A chỉ xuất hiện đúng một ảnh, đúng loại thứ mà trung bình hoá nhiều ảnh chồng phủ sẽ dập tắt.

Hai chỗ phải nói kèm, cả hai đều nghiêng theo hướng có lợi cho kết luận trên:

- **P 0,962 là cận trên.** 29 tấm nhánh B khoanh mà chưa ai soi; nếu cả 29 đều sai thì P còn 0,896. Nhánh A không có khoảng mờ này (0 tấm chưa soi) nên 0,865 của nó là số chắc. Dù lấy cận dưới thì nhánh B vẫn hơn về precision.

- **Bộ nhãn thiên vị nhánh A**, vì người soi chấm chủ yếu trên tấm mà nhánh A đã khoanh. Thiên vị đó làm recall của A đẹp lên và của B xấu đi — nên 0,860 kia là số bị ép xuống, không phải bị thổi lên.

---

9. Sai số định vị

RMSE end-to-end	**2,56 cm** (11 điểm đối chứng tay, ảnh 0280)
Ngưỡng đề bài	30 cm — dư hơn 10 lần
GSD ortho màu	0,957 cm/px, tức sai số ≈ 2,7 px
GCP trong `cameras.xml`	**0**

Ba điều phải nói kèm khi trích con số này:

1. **Đây là sai số tương đối.** 2,56 cm là độ khớp giữa ortho nhiệt và ortho màu — hai sản phẩm dựng từ *cùng một khối ảnh*. Nó nói rằng chuỗi chiếu đúng, không nói rằng toạ độ tuyệt đối đúng.
2. **Nhưng nền tuyệt đối tốt hơn vẻ ngoài.** `cameras.xml` ghi `accuracy_cameras = 10 m` — đó chỉ là **mặc định Metashape chưa ai sửa**, không phải phép đo. Thực tế 598/598 ảnh có `RtkFlag = 50` (RTK fix) với độ lệch ngang 0,83 cm. Chuyến bay này **là RTK**.
3. **Con số 20 cm từng lưu hành là sai** — nó đến từ một phép đo khác đại lượng, không phải từ chuỗi này.

---

10. Kết luận

Lý do vẫn ghi trong tài liệu cũ — *"ghép ortho làm nhoè hotspot nên phải detect trước"* — **đã bị chính phép đo bác bỏ** (mục 7.3). Vẫn chọn nhánh A, nhưng vì ba lý do khác:

1. **Recall** — 0,968 so với 0,860. Với bảo trì, một lỗi bỏ sót đắt hơn một lần đi kiểm tra oan, nên recall là trục ưu tiên, và đó là trục nhánh A thắng.
2. **Vùng ngoài ortho** — 14/603 lỗi chỉ nhìn thấy được từ ảnh không vào ortho. Nhánh B *không thể* tìm ra chúng, bất kể model tốt đến đâu.
3. **Bảng màu gốc** — nhánh A cho model ăn ironbow gốc; ortho hiện dựng từ ảnh đã chuyển xám nên mất 10,4 điểm ngay từ đầu vào.

Khoảng cách thật là khoảng 17 điểm, không phải 47. Đủ để giữ lựa chọn hiện tại, nhưng *không* đủ để viết rằng ortho phá hỏng tín hiệu nhiệt. Ba hướng thu hẹp khoảng cách đó, đều chưa làm:

- dựng lại ortho **giữ 3 kênh ironbow** thay vì chuyển xám — lấy lại phần lớn 10,4 điểm;
- ghép bằng **max** thay vì trung bình — hotspot là giá trị cực trị, trung bình hoá là phép sai với nó;
- ghép trên **dữ liệu bức xạ 16-bit trong APP3**, hiện đang bỏ phí — cho ortho một ý nghĩa vật lý thay vì chỉ là ảnh JPEG 8-bit ghép lại.

Phụ lục A — Các con số không được trích

con số	vì sao không dùng được
mAP@50 = 0,965	Đo trên tập test mà **81% ảnh nằm trong train** của model (lệch bộ chia). Số trên tập held-out thật là 0,974; kỳ vọng ở trang trại chưa từng bay khoảng **0,70**.

con số	vì sao không dùng được
Recall = 0,897	Là số ở điểm tối ưu F1 (conf 0,64), không phải ở ngưỡng chạy thật 0,25. Đo lại bằng ghép cặp trực tiếp thì ở conf 0,25 là **97,5%**.
Đúng lớp = 97,2%	Công cụ gán nhãn **hiện sẵn nhãn model gợi ý** và Enter = đồng ý. Phần lớn con số này đo tần suất bấm Enter. Ranh giới Single↔Multi HotSpot đúng là loại ca dễ gật cho qua nhất.
Recall = 0,968 (nhánh A)	Là **cận trên**, thiếu bằng chứng độ phủ — người soi chủ yếu soi tấm model đã khoanh nên FN thấp giả tạo. Mẫu 400 tấm ngẫu nhiên đã bốc nhưng **chưa soi**.

Số dùng được: Precision 0,865 của nhánh A (người soi bác model 101/519 lần = 19,5%, không phải mash Enter) · RMSE 2,56 cm · và toàn bộ bảng ablation ở mục 7, vì nó so cùng một model với chính nó.

Phụ lục B — Script và file kết quả

Chạy lại toàn bộ

```
py -3.13 ma/calib_thermal_rgb.py     # chỉ một lần cho mỗi model drone
py -3.13 ma/loc_anh_tre.py           # ~45 phút
py -3.13 ma/loc_anh_rung.py          # 3 giây
py -3.13 ma/ghep_ortho_nhiet.py      # -> ortho_nhiet_tu_lam.tif
py -3.13 ma/quet_loi_anh_le.py       # -> loi_tu_anh_le.json      (nhánh A)
py -3.13 ma/gan_tam_pin_4lop.py      # -> loi_4lop_cuoi.json      ĐẦU RA CHÍNH
py -3.13 ma/danh_gia_chi_so.py
```

Chạy lại phép so hai nhánh

```
py -3.13 ma/thu_detect_tren_ortho.py --xoay 270      # ablation A/B/C/D
py -3.13 ma/vi_sao_ortho_kem.py                      # nét vs tương phản
py -3.13 ma/va_viewer_ortho_xoay.py                  # thêm bộ vào viewer so sánh
```

File kết quả

file	nội dung
`ket_qua/loi_4lop_cuoi.json`	**603 lỗi, đầu ra chính** — nhánh A, đã gán mã tấm
`ket_qua/loi_ortho_xoay270_cuoi.json`	463 lỗi — nhánh B, đã gán mã tấm
`ket_qua/ablation_frame_vs_ortho.json`	bốn điều kiện, bản ortho bắc-lên
`ket_qua/ablation_frame_vs_ortho_xoay270.json`	bốn điều kiện, bản ortho xoay 270°
`ket_qua/vi_sao_ortho_kem.json`	độ nét cấu trúc và tương phản nhiệt
`ket_qua/danh_gia_theo_nhan.json`	P/R/F1 nhánh A trên nhãn tay
`ket_qua/danh_gia_theo_nhan_ortho.json`	P/R/F1 nhánh B trên nhãn tay
`ket_qua/sai_so_end_to_end.json`	RMSE 2,56 cm, 11 điểm đối chứng
`cong_cu/so_sanh_model.html`	viewer 4 bộ, gạt qua lại để so trực quan